

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о командном программном проекте на тему:
Разработка онлайн бизнес-игры “Мультикомандный Featureban”

Выполнили студенты:

группы #БПМИ244, 2 курса
группы #БПМИ245, 2 курса
группы #БПМИ246, 2 курса
группы #БПМИ249, 2 курса
группы #БПМИ249, 2 курса
группы #БПМИ2411, 2 курса

Бакунц Арина Александровна
Денисов Николай Дмитриевич
Овчаров Иван Евгеньевич
Мостова Александра Александровна
Сухобок Тимофей Михайлович
Чупрына Матвей Михайлович

Принял руководитель проекта:

Коротков Александр Олегович
Руководитель группы, группа дополнительных продуктов
АО "Т-Банк"

Содержание

1	Аннотация	4
2	Введение	5
3	Обзор литературы	7
4	Процессы разработки	10
4.1	Сравнительный анализ методологий управления разработкой	10
4.2	Обоснование гибридного подхода: Асинхронный пайплайн	10
4.3	Жизненный цикл задачи	11
4.4	Визуализация и контроль потока	11
5	Обеспечение качества и роль QA в проекте	13
5.1	Организация QA-процесса	13
5.1.1	Shift-left подход и тест-аналитика на этапе формирования требований .	13
5.1.2	Контроль качества в ходе разработки	13
5.1.3	Основные практики QA и тестирования	14
5.2	Тестирование и стратегия	15
5.2.1	Тест-дизайн и управление рисками	15
5.2.2	Автотесты backend	15
5.2.3	Структура тестирования	16
5.2.4	Примеры автоматизированных сценариев	16
5.2.5	Окружения и организация тестирования	17
5.2.6	Критерии входа и выхода из тестирования	17
5.2.7	Нефункциональные проверки	17
5.3	Работа с багами	18
5.3.1	Подход и итоги	18
5.3.2	Классификация	18
5.3.3	Организация работы и тестирование	18
5.4	Ручное функциональное тестирование	19
5.4.1	Цели и границы ручного тестирования	19
5.4.2	Организация ручного тестирования	19
5.4.3	Фиксация результатов и работа с дефектами	20
5.4.4	Ключевые выводы по ручному тестированию	20

6	Работа над проектом и распределение задач по участникам	21
6.1	Этапы реализации проекта	21
6.2	Функциональная декомпозиция системы	21
6.3	Распределение задач по участникам	22
7	Техническая документация	24
7.1	Проектирование инфраструктуры и регламентация процессов управления исходным кодом	24
7.1.1	Организация репозиторий	24
7.1.2	Стратегия управления ветвлением	25
7.1.3	Технологический стек и автоматизация	27
7.1.4	Процесс релиза	27
7.2	Страница лендинга	27
7.2.1	Сетевые маршруты и протоколы	27
7.2.2	Архитектура клиентского приложения (Frontend)	29
7.2.3	Архитектура серверной части (Backend)	29
7.2.4	Синхронизация состояния: WebSocket и механизмы отказоустойчивости	30
7.3	Архитектура и логика страницы игровой доски (Kanban)	32
7.3.1	Клиентская архитектура (Frontend)	32
7.3.2	Структура игрового пространства	32
7.3.3	Распределенное состояние и отказоустойчивость синхронизации	33
7.4	Игровой цикл и механика непрерывного улучшения	33
7.4.1	Инициализация и запуск	33
7.4.2	Ежедневная продуктовая разработка	34
7.4.3	Ретроспектива и подстройка процесса	34
7.4.4	Завершение игры	35
8	Описание применения генеративной модели	36
9	Заключение	37
	Список литературы	38

1 Аннотация

Проект посвящен созданию веб-версии бизнес-игры "Мультикомандный Featureban". Игра моделирует процесс разработки. Освоение командной работы и Канбан-методов управления задачами происходит у участников непосредственно в ходе игры [20].

Очный вариант игры уже опробован на множестве команд и показал хорошие результаты. Однако очный формат ограничивает масштабируемость: он требует физического присутствия и усложняет работу модератора. Перевод игры в онлайн снимает проблему поиска помещения и облегчает сбор команды.

Результатом работы стало веб-приложение, поддерживающее синхронную работу нескольких команд. В системе предусмотрен удобный интерфейс для управления задачами и совершения ходов. Ведущему доступны специальные инструменты для управления ходом игры и контроля процесса.

Ключевые слова

Канбан, симуляция, бизнес-игра, Фичебан, мультикомандный, корпоративные тренинги, веб-приложение, закон Литтла

2 Введение

В условиях современного развития информационных технологий и усложнения программных продуктов существенно возрастает сложность процессов их разработки и сопровождения. Современные компании, особенно в сфере информационных технологий, все чаще переходят от работы отдельных автономных команд к модели, в которой несколько команд параллельно развивают один продукт или платформу. Такая организация труда требует более высокого уровня координации, прозрачности процессов и согласованности действий между участниками.

В этих условиях особое значение приобретают гибкие методологии управления проектами, ориентированные не только на локальную эффективность отдельных команд, но и на оптимизацию всей системы в целом. Одной из наиболее распространенных и практически применяемых методологий является Kanban. В отличие от итеративных подходов, таких как Scrum, Kanban ориентирован на непрерывный поток работ, визуализацию процессов и ограничение количества незавершенных задач. Данный подход хорошо масштабируется и широко используется в организациях со сложной структурой и уже существующими продуктами.

При этом ключевые принципы Kanban, включая ограничение WIP (Work in Progress), управление потоком и анализ метрик производительности, достаточно сложно усваиваются при изучении исключительно теоретических материалов. Без практического опыта участникам трудно осознать влияние управленческих решений на пропускную способность системы, время выполнения задач и стабильность процессов. В результате между формальным знанием методологии и ее реальным применением на практике возникает существенный разрыв.

Для преодоления данного разрыва в корпоративном обучении и образовательных программах широко применяются бизнес-игры и игровые симуляции. Они позволяют участникам в безопасной среде экспериментировать с процессами, принимать управленческие решения и наблюдать их последствия без риска для реальных проектов. Игровой формат способствует более глубокому пониманию системных эффектов, возникающих при управлении потоком работ, и позволяет наглядно продемонстрировать влияние ограничений и изменений в процессе разработки.

Одной из таких симуляций является игра Featureban. Эта игра позволяет смоделировать процесс создания нового продукта с использованием Kanban-подхода. Классическая версия игры ориентирована на работу одной команды и позволяет продемонстрировать базовые принципы управления потоком. Мультикомандная версия Featureban представляет собой развитие данной симуляции и направлена на моделирование взаимодействия нескольких

команд, одновременно работающих над общим продуктом. В рамках такой игры проявляются межкомандные зависимости, узкие места системы, проблемы синхронизации и эффекты локальной оптимизации, характерные для реальных крупных проектов.

Актуальность данной работы обусловлена возрастающей сложностью управления процессами разработки в условиях масштабирования продуктов и увеличения количества команд, вовлеченных в совместную работу. На практике многие организации сталкиваются с тем, что методы и практики Kanban, успешно применяемые на уровне одной команды, теряют эффективность при переходе к мультикомандному формату. При этом существующие инструменты и обучающие материалы не позволяют в полной мере продемонстрировать системные эффекты, возникающие при межкомандном взаимодействии, и обеспечить корректное воспроизведение игровой механики мультикомандной симуляции.

В этой связи актуальной является задача создания специализированного программного инструмента, позволяющего наглядно моделировать мультикомандные процессы разработки, демонстрировать влияние управленческих решений на всю систему в целом и обеспечивать автоматизацию игровых правил и сбора метрик. Решение данной задачи способствует повышению эффективности обучения и более осознанному внедрению Kanban-подходов в организациях со сложной структурой.

Целью данной курсовой работы является разработка онлайн-версии бизнес-игры «Мультикомандный Featureban», предназначенной для моделирования межкомандного взаимодействия и использования в обучающих и тренинговых форматах. Для достижения поставленной цели в рамках работы проводится анализ предметной области, формулируются требования к разрабатываемой системе, проектируется архитектура веб приложения и реализуются основные компоненты системы.

В результате планируется создать веб приложение, поддерживающее проведение игровых сессий с участием нескольких команд, разграничение ролей пользователей, визуализацию процессов и автоматизированный сбор ключевых метрик игрового процесса. Новизна данной работы заключается в разработке специализированного инструмента для онлайн-проведения мультикомандной версии Featureban, который учитывает особенности игровой механики, поддерживает сценарии симуляции и обеспечивает централизованное управление игровым процессом. Практическая значимость работы состоит в возможности использования разработанного приложения для проведения корпоративных тренингов и образовательных мероприятий, направленных на изучение принципов управления потоком и межкомандного взаимодействия.

3 Обзор литературы

Среди методов управления самыми популярными являются Scrum и Kanban. Идея методологии Scrum - разработка итерациями. То есть за определенный промежуток времени команда должна выпустить часть готового продукта. Scrum традиционно применяют при создании нового продукта. Kanban, напротив, чаще используют в крупных компаниях с уже работающим продуктом и множеством команд. Этот метод сфокусирован на управлении потоками работ и ограничении незавершенных работ.

Принципы гибких методов управления сложно усвоить исключительно с помощью книг и статей. Эта проблема подтверждается результатами исследований в области менеджмента и бизнес-образования. Например, обзор *The effectiveness of games for educational purposes: A review of recent research* [2] подтверждает: игровые методы и симуляции развивают управленческие навыки лучше, чем лекции или чтение литературы. Это связано с необходимостью понимать внутренние процессы системы и их взаимосвязь.

В первую очередь это касается методологий Kanban [20] и Lean [19]. Поэтому в обучении широко применяются игровые симуляции. В игре участники могут тестировать любые решения и сразу видеть их последствия, не боясь навредить реальному проекту. Эффективность этого метода подтверждается теорией из книги Д. Колба *Experiential Learning Cycle* [1]. Согласно его модели, глубокое обучение проходит через четыре этапа: накопление конкретного опыта, рефлексивное наблюдение, абстрактную концептуализацию и активное экспериментирование. Бизнес-игра позволяет быстро пройти этот цикл, закрепляя ценные практические навыки.

В работе *Learning with a strategic management simulation game: A case study* [13] авторы также изучали эффективность бизнес-симуляций. Исследователи делают вывод, что игры помогают увидеть картину целиком и использовать теорию в деле. Так проще понять связи внутри процесса, чем при изучении обычных учебных материалов.

Рассмотрим подробнее эти подходы. В своей книге Дэвид Андерсон определяет шесть базовых практик метода Kanban [3]:

- 1 Визуализация потока задач.
- 2 Ограничение объема незавершенной работы (WIP).
- 3 Непосредственное управление потоком.
- 4 Четкие правила выполнения работы.
- 5 Регулярные встречи для сбора обратной связи.

6 Совместное улучшение процесса.

а также подчеркивает принцип: *start with what you do now*. В игровой форме участникам наглядно видно связь принятых решений в управлении и скорость доведения задач до конца. Этот метод тесно связан с концепцией Lean. Целью обоих методов является устранение потерь и увеличение пропускной способности. Практическая часть опирается на Теорию ограничений Э. Голдратта [9]. Согласно ей, общая производительность системы лимитируется её "узкими местами". Эти принципы легли в основу разрабатываемой симуляции, моделирующей реальные процессы для обучения.

Также в управлении проектами часто используют закон Литтла [21]. На практике этот закон позволяет менеджерам проектов точнее прогнозировать сроки и понимать реальную скорость работы команды. В методологии Kanban закон выражается формулой:

$$L = \lambda \times W \quad (1)$$

где L - это незавершенная работа (количество активных задач в системе), λ - пропускная способность (количество выполненных задач за единицу времени), W - время цикла (время прохождения задачи через процесс). Однако, прочитав, сложно понять, как она работает на самом деле, людьми эта формула воспринимается абстрактно. Игровая симуляция дает людям понять на примере, как действует этот закон, и позволяет увидеть, какой результат дают ограничения на WIP.

На данный момент есть онлайн-версия игры Featureban¹, но эта версия игры только для одной команды и рассчитана на 3–5 человек. Версия игры на несколько команд существует только в оффлайн формате и разработана сертифицированными экспертами из России (Neogenda)[22]. где L - это незавершенная работа (количество активных задач в системе), λ - пропускная способность (количество выполненных задач за единицу времени), W - время цикла (время прохождения задачи через процесс). Однако, прочитав, сложно понять, как она работает на самом деле, людьми эта формула воспринимается абстрактно. Игровая симуляция дает людям понять на примере, как действует этот закон, и позволяет увидеть, какой результат дают ограничения на WIP.

На данный момент есть онлайн-версия игры Featureban², но эта версия игры только для одной команды и рассчитана на 3–5 человек. Версия игры на несколько команд существует только в оффлайн формате и разработана сертифицированными экспертами из России

¹Онлайн версия игры доступна по ссылке: <https://tools.flowfast.io/featureban>, дата обр. 13.01.2026

²Онлайн версия игры доступна по ссылке: <https://tools.flowfast.io/featureban>, дата обр. 13.01.2026

(Neogenda)[22]. Исходная версия Featureban от Майка Барроуза доступна под лицензией Creative Commons Attribution ShareAlike 4.0 [4]. Все материалы разрабатываемой мультикомандной модификации наследуют этот же правовой статус. Сейчас для проведения игры с несколькими командами приходится адаптировать сторонние платформы, например, Migo или Trello. Однако данные решения не могут быть напрямую использованы для реализации симуляции.

Во-первых, они только визуализируют игру, но не накладывают никаких ограничений на действия участников. Из-за этого пропадает поддержка ролей, разделение на команды, разрушаются четкие сценарии игр, что являются критически важными аспектами симуляции для корректного моделирования принципов Kanban. Исследования игровых симуляций подчеркивают, что строгость правил, ограничений и сценариев является критически важным аспектом подобных обучающих методов.

Во-вторых, подобные платформы не собирают нужную статистику. При отсутствии автоматизации сбора статистики увеличивается влияние человеческого фактора, становится сложнее проводить рефлекссию над экспериментом, что приводит к снижению педагогической ценности проведения этой игры.

Таким образом, такие универсальные инструменты не предоставляют достаточно функциональный интерфейс для высокоэффективного проведения мультикомандной версии игры.

4 Процессы разработки

4.1 Сравнительный анализ методологий управления разработкой

Для определения оптимального пути реализации проекта были рассмотрены три доминирующих парадигмы в индустрии программного обеспечения:

- **Каскадная модель (Waterfall):** Подход, при котором все требования обговариваются до начала работы. Несмотря на предсказуемость, метод был отвергнут, потому что была высокая степень неопределенности продукта. В разработке сайта нужно иметь возможность интерфейс на основе первых тестов, что невозможно в жестких рамках Waterfall.
- **Scrum-фреймворк [17]:** Итеративный подход, который ориентирован на кросс-функциональность. Команда отказалась от "чистого" Scrum, так как в условиях становления команды и наличия ярко выраженной специализации (выделенная роль аналитика) возникал риск неэффективного планирования и простоев разработчиков в ожидании документации в начале спринта.
- **Kanban-метод [3]:** Система управления потоком, минимизирующая незавершенное производство (WIP). Несмотря на использование элементов Kanban в самой тематике нашего продукта, для внутренней разработки метод показался недостаточно структурированным для новой команды, которой необходим спринты для поддержания темпа.

4.2 Обоснование гибридного подхода: Асинхронный пайплайн

В результате анализа был сформирован гибридный метод, сочетающий дисциплину Scrum (итерации) и принципы управления потоком. Суть метода заключается в разделении жизненного цикла задачи на два параллельных трека:

Аналитика: Опережающая работа аналитика. Пока команда разработки занята текущей итерацией, аналитик готовит документацию, схемы API и требования для следующей "Стори". Это позволяет нивелировать риск "пустого бэклога" на старте итерации.

Разработка: Реализация согласованного объема задач разработчиками в фиксированные циклы от 1 до 3 недель. Данная схема научно обоснована Теорией ограничений Э. Голдратта [9]. Выделяя аналитику в отдельный этап, мы расширяем "узкое горлышко" системы, обеспечивая максимальную пропускную способность команды разработки.

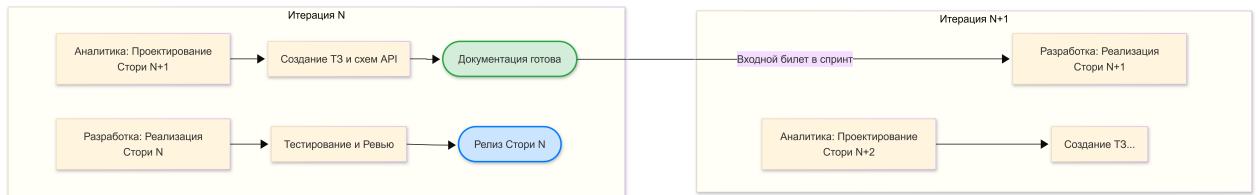


Рис. 4.1: Схема асинхронного пайплайна гибридного процесса разработки

4.3 Жизненный цикл задачи

Работа над каждой стори в нашей команде делится на несколько последовательных шагов:

- 1 **Аналитика:** Проработка ролевых моделей, построение схемы взаимодействия с API, продумывание дизайна при Front-end разработке. Здесь происходит декомпозиция стори с учетом процесса разработки.
- 2 **Встреча 3-Amigos:** Короткий созвон, на которой аналитик (один из разработчиков по очередности) презентует документацию. На этом этапе планируется работа и распределяются задачи между Front-end, Back-end и DevOps. Это позволяет прояснить все вопросы до старта разработки.
- 3 **Реализация:** Разработка, покрытие юнит-тестами и настройка мониторинга/аллертов.

Подобное разделение опирается на "Lean Thinking-[19]. Это позволяет не терять время на выяснение деталей и снижает количество ошибок в требованиях.

4.4 Визуализация и контроль потока

Чтобы наглядно контролировать разработку, в проекте используется электронная канбан-доска. Она построена под наш гибкий метод и показывает, как каждая задача проходит четыре этапа:

- 1 **Todo:** этап формирования и приоритезации бэклога.
- 2 **In progress:** активная фаза реализации. Для задач аналитики этот этап подразумевает исследование и создание документации. Для задач разработки - написание кода на основе уже готовых требований.
- 3 **Review:** этап верификации. В контексте аналитики этот статус означает проведение встречи, на которой аналитик представляет свое решение. В контексте разработки - проведение код-ревью и контроль качества.

4 **Done:** задача проверена, соответствует критериям качества и идет в релиз.

Особое внимание уделяется правилам перехода между этапами. Разработчики берут стори в работу только после полного завершения аналитики. Это исключает старт разработки по сырым требованиям и поддерживает ровный поток задач. Именно такой подход диктуют закон Литтла [21] и теория ограничений [9].

Все процессы были рассмотрены и утверждены командой в качестве стандартов коммуникации.

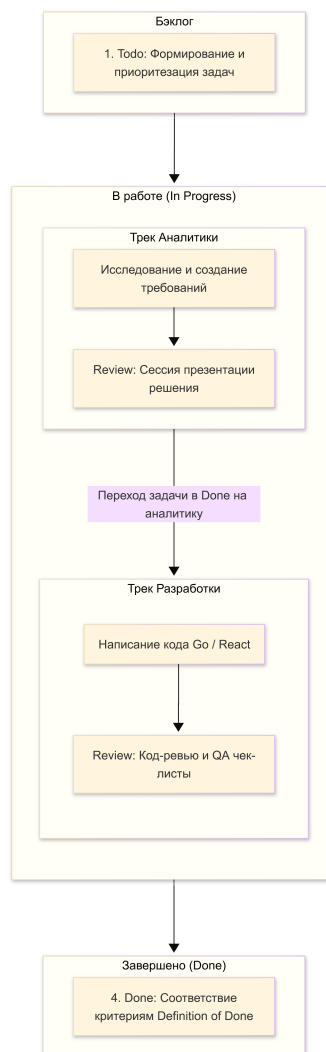


Рис. 4.2: Этапы прохождения функциональной единицы (Стори) по Канбан-доске

5 Обеспечение качества и роль QA в проекте

В рамках проекта процесс управления качеством рассматривался как непрерывный процесс, а не как разовая проверка перед демонстрацией. Роль контроля качества заключалась в поддержании соответствия между концепцией продукта, технической реализацией и фактическим поведением пользователя в системе. На практике это проявляется в мониторинге требований, разработке стратегий тестирования, постоянном анализе сценариев риска и поддержании цикла запуска в эксплуатацию.

5.1 Организация QA-процесса

5.1.1 Shift-left подход и тест-аналитика на этапе формирования требований

Работа по качеству началась еще до разработки. В начале проекта были выставлены приемлемые критерии к продукту, которые могут быть проверены.

Ключевые задачи QA на этом этапе были следующими:

- формализация границ MVP и исключение второстепенных функций, не влияющих на основной игровой цикл;
- выделение ролей (ведущий, игрок, команда) и ограничений их действий в API;
- декомпозиция всего игрового процесса на проверяемые шаги;
- фиксация рисков, которые могли привести к срыву сессии;
- согласование Definition of Ready для задач, чтобы в разработку попадали только проверяемые Стори.

Проделанная работа позволила сократить количество новых задач в середине работы команды, когда реализация технически завершена, но не соответствует исходному пользовательскому ожиданию.

5.1.2 Контроль качества в ходе разработки

На этапе реализации QA-процесс был встроен в ежедневный ритм команды. Контролировалось, чтобы каждая задача проходила минимальный набор этапов: уточнение критериев, реализация, самопроверка разработчика, проверка по чек-листу, регрессионный прогон перед слиянием.

Для этого применялись следующие практики:

- 1 ведение тест-дизайна на уровне сценариев и негативных кейсов одновременно с декомпозицией backend-задач;
- 2 проверка изменений на совместимость с уже реализованными фазами игры `setup`, `running`, `retro`, `finished`;
- 3 контроль API-контрактов (статусы, структура JSON, тексты ошибок) для предотвращения скрытых интеграционных дефектов frontend/backend;
- 4 контроль инвариантов доменной логики: доступность действий только соответствующим ролям, корректное обновление досок команд, неизбыточное создание сущностей;
- 5 регрессионная проверка рискованных мест после каждого изменения логики хода дня и управления WIP-лимитами.

5.1.3 Основные практики QA и тестирования

В проекте использовались следующие практики, обеспечивающие воспроизводимость и контроль качества:

- **STLC в облегченном виде:** постановка требований → тест-дизайн → подготовка тестовых данных → выполнение проверок → фиксация дефектов и регрессия.
- **Риск-ориентированное тестирование:** приоритизация сценариев, которые могут сорвать игровую сессию (переходы фаз, роли, целостность досок).
- **Контрактные проверки API:** согласование структуры JSON и кодов ошибок как входных критериев приемки задач.
- **Definition of Ready/Done:** задачи берутся в работу только при наличии критериев приемки; завершение допускается после проверки критичных кейсов.
- **Code review и самопроверка:** обязательная быстрая проверка основных сценариев разработчиком перед слиянием.
- **Инженерные принципы KISS/DRY/SOLID:** отсутствие дублирования ранее написанного функционала.

5.2 Тестирование и стратегия

5.2.1 Тест-дизайн и управление рисками

Мы сократили общее количество проверок, оставив фокус на наиболее важных рисках. Для API применяли методы классов эквивалентности и анализа граничных значений (проверяли, например, корректность WIP и длину ID). Поведение игровых фаз контролировали через таблицы решений и моделирование состояний. Это дало возможность детально проработать те сценарии, где ошибки обходятся дороже всего. Это позволило сосредоточиться на сценариях, где вероятность и цена ошибки максимальны.

Риски фиксировались в виде короткого списка контрольных точек и регулярно пересматривались. Типовые риски:

- невалидные переходы между этапами игры (`setup`, `running`, `retro`);
- рассинхронизация состояния между командами и сервером при параллельных действиях;
- нарушения ролевой модели (действия не-ведущим);
- потеря корректности доски при перемещении карточек и ограничениях WIP.

5.2.2 Автотесты backend

Бэкенд покрыт тестами на `pytest` в три этапа:

- **Unit** (`tests/unit/test_backend_api.py`) - быстрая валидация бизнес-логики отдельных методов;
- **WebSocket** (`tests/unit/test_websockets.py`) - контроль формата данных и устойчивости сокета;
- **E2E** (`tests/e2e/test_game_flow.py`) - полный прогон игрового сценария, как это делает живой пользователь.

На уровне `unit` мы проверяем ответы API на правильные и ошибочные запросы. Тесты веб-сокетов проверяют стабильность подключения и правильность формата данных, в том числе при обрывах связи. E2E-уровень проверяет устойчивость всей цепочки действий пользователя: от подготовки сессии до перехода в ретро и продолжения игрового цикла.

Маркеры `unit` и `e2e` в `pytest.ini` позволяют запускать тесты выборочно. Перед слиянием кода разработчики и тестировщики запускали эти проверки локально.

5.2.3 Структура тестирования

В процессе проверки использовались наборы тест-кейсов, разбитые по конкретным сценариям:

- **Жизненный цикл игры:** корректное выполнение функций, отвечающих за цикл игры;
- **Ролевая модель:** проверка доступности функций в разных ролях;
- **Проверка валидаций:** проверки ограничений team/game ID, проверка диапазонов значений полей и валидации на обязательность;
- **Согласованность командных досок:** синхронизация действий между разными командами;
- **Фазовые ограничения:** проверка условий для изменения WIP допускается.

Благодаря этому мы исключили критические баги в логике игры и упростили регрессионное тестирование при добавлении новых функций.

5.2.4 Примеры автоматизированных сценариев

Ниже приведены примеры автотестов, отражающие ключевые риски:

- **Контракт создания игры:** проверка начальных настроек (`test_create_game_returns_expected_structure`).
- **Ролевая модель:** Проверка прав доступа: Обычные участники не могут начать проект или игру, это делает только ведущий (`test_start_project_forbidden_for_non_facilitator`, `test_start_game_forbidden_for_non_facilitator`).
- **Фазовые ограничения:** Нельзя перетащить карточку, пока игра не началась или когда идет ретроспектива (`test_drag_before_game_started`, `test_drag_in_retro_phase`).
- **Сквозной цикл:** переходы по дням, ретро-фаза, изменение WIP и продолжение игры (`test_full_game_cycle_to_retro_and_continue`).
- **WebSocket-контракты:** Проверяем ответы сервера на ping/pong и на сообщения с неизвестным типом `test_lobby_ws_ping_pong`, `test_lobby_ws_unknown_type`.

5.2.5 Окружения и организация тестирования

Тестирование выполнялось в двух окружениях. Тесты запускали локально у разработчиков и QA-инженеров, а также на сервере, развернутом через `docker-compose`, чтобы проверить полные сценарии работы пользователей. Основной фреймворк автоматизации — `pytest`; отчеты о покрытии не собирались, приоритетом были сценарии поведения и инварианты доменной логики. Перед показами мы вручную проверяли главные маршруты и запускали unit-тесты.

5.2.6 Критерии входа и выхода из тестирования

Релизный кандидат считался готовым, если выполнялись простые условия:

- **Вход:** определены критерии приемки, сценарии MVP описаны, изменения не ломают существующие API-контракты;
- **Выход:** пройдены smoke- и e2e-тесты, критические ошибки отсутствуют. Стабильность проверена на сценариях: создание игры, запуск, ход дня, ретро.

Это исключало выпуск релиза с багами, ломающими игру.

5.2.7 Нефункциональные проверки

Несмотря на учебный формат, мы уделили внимание базовым нефункциональным характеристикам:

- **Надежность:** проверка корректной обработки ошибочных запросов и невозможных действий;
- **Производительность:** контроль отсутствия деградаций в основных API-вызовах при увеличении числа участников;
- **Совместимость:** проверка корректной работы через веб-клиент и стабильность WebSocket-соединений;
- **Восстанавливаемость:** проверка поведения клиента при переподключениях и повторной синхронизации состояния.

5.3 Работа с багами

5.3.1 Подход и итоги

Баг-трекера не было, поэтому ошибки регистрировали как обычные задачи, указывая их важность в комментариях. Приоритет назначали вручную, отдельно выделяя критические сбои, ломающие геймплей или UX.

Во время общего ручного тестирования команда нашла около 5 блокирующих ошибок и столько же важных. Критические правки делали обязательно перед демо, остальные уходили в бэклог на доработку.

5.3.2 Классификация

Для быстрой сортировки проблем мы использовали простую шкалу:

- **Blocker**: невозможность провести игровую сессию или запустить ключевую фазу;
- **Critical**: нарушение логики игры или потеря консистентности состояния;
- **Major**: заметное ухудшение UX или стабильности, не блокирующее игру полностью;
- **Minor**: косметические дефекты и улучшения, не влияющие на сценарий.

Мы оценивали важность задачи по трем критериям: влияние на геймплей, частота появления бага и срочность перед ближайшим демо.

5.3.3 Организация работы и тестирование

Мы следили не только за тестами, но и за ходом работы команды. Это помогало выпускать релизы без неприятных сюрпризов.

- соблюдение Definition of Done перед переносом задачи в *Done*;
- прозрачный учет дефектов и разделение их по приоритету (блокирующие игровой сценарий, критичные по UX, некритичные);
- проверка готовности сборки к демо по чек-листу (стабильность основных маршрутов, отсутствие критичных ошибок в сценарии занятия);
- участие в ретроспективах с акцентом на причины дефектов процесса: позднее уточнение требований, неполное покрытие пограничных состояний, отсутствие регрессии в рискованных точках;

- формирование предложений по улучшениям: раннее уточнение критериев, расширение негативных тестов и фиксация API-контрактов до начала реализации.

5.4 Ручное функциональное тестирование

5.4.1 Цели и границы ручного тестирования

Кроме автоматизированных проверок в проекте использовалось ручное функциональное тестирование. Оно нужно, чтобы покрывать сценарии неочевидных взаимодействий, оценку совместимости и качества пользовательского интерфейса, а также в качестве последней проверки перед демонстрацией, позволяющей выявить дефекты, пропущенные автоматизированными методами из-за особенностей тестового окружения. Ручное тестирование в проекте применялось для:

- проверки пользовательского опыта (UX) и визуального соответствия требованиям;
- оценки читаемости логов, понятности сообщений об ошибках, навигационной интуитивности;
- исследования системы в поиске непредусмотренных сценариев, багов и аномалий;
- проверки интеграции фронтенда и бэкенда в режиме реального времени при параллельных действиях нескольких участников;
- оценки совместимости системы с разными браузерами и устройствами;
- smoke-тестирования сборок перед демонстрациями и встречами.

5.4.2 Организация ручного тестирования

Процесс ручного тестирования был встроен в ритм команды и включал следующие регулярные активности:

- 1 **Проверка критического пути:** после каждого значимого изменения тестировщик выполнял smoke-прогон по ключевым сценариям (создание игры, подключение нескольких клиентов, проведение полной игровой сессии). Это позволяло оперативно выявлять серьезные проблемы до того, как они закрепятся в коде.
- 2 **Исследовательские сессии:** на встречах команда проводила тестовые игры, в ходе которых разработчики пытались "сломать" систему: совершать действия в неожиданном

порядке, одновременно от разных ролей, с некорректными или пустыми данными. Далее тестировщиком составлялся список проблем и доработок, ранжированный по степени важности.

- 3 **Ручная регрессия перед демонстрациями:** за день до плановой демонстрации тестировщик выполнял полную проверку утвержденного чек-листа, включавшего основные пользовательские маршруты, ролевую модель, фазовые переходы, негативные сценарии.

5.4.3 Фиксация результатов и работа с дефектами

Каждый выявленный дефект записывался в следующем формате:

- **Фактическое поведение:** что произошло;
- **Ожидаемое поведение:** что должно происходить;
- **Шаги воспроизведения:** последовательность действий, приводящая к дефекту;
- **Приоритет:** критическая проблема или мелкая рекомендация.

Такая практика позволяла команде быстро локализовать и устранить неполадки.

5.4.4 Ключевые выводы по ручному тестированию

Опыт проекта показал, что ручное тестирование остается необходимым компонентом QA-стратегии даже при наличии широкого покрытия автотестами. Оно эффективно закрывает слепые зоны автоматизации: визуальные несоответствия, человеческий фактор, пользовательский опыт, многопользовательские сценарии. При этом регулярный ритм проверок и формализованные чек-листы делают ручное тестирование воспроизводимым и измеримым, позволяя оценивать прогресс и зоны риска.

6 Работа над проектом и распределение задач по участникам

6.1 Этапы реализации проекта

В рамках курсового проекта был применен метод итерационной разработки, предполагающий поэтапное выдвижение требований и разработки функциональности. Первоначально был выполнен выбор технологического стека и реализована MVP-версия проекта, содержащий базовый функционал проекта:

- Главное меню игры с возможностью сформировать команды, создать команды или создать игру;
- Командные и общие доски;
- Карточки проектов и задач с функционалом блокировки и перетаскивания по доске;
- Логика подбрасывания монетки и хода игрока;
- Интерфейс для фасилитатора игры;
- Генерация карточек с проектами;
- Поддержка одной игровой сессии;
- Развертывание базовой инфраструктуры для функционирования игровой системы.

Затем система дополнялась расширенными возможностями:

- Повышение надежности системы;
- Организация ретро-фазы обсуждения между игроками;
- Доработка пользовательского интерфейса.

6.2 Функциональная декомпозиция системы

Проектируемая система включает основные функциональные составляющие:

- **Игровая логика**
 - сценарий проведения игры;

- этапы игры и переходы между ними;
- сбор и расчет игровых метрик (WIP, lead time и др.).

- **Пользовательские роли**

- ведущий (модерация игры, управление процессом);
- команда (совместное принятие решений);
- игрок (участие в игровых активностях);
- наблюдающий (видит ход игры).

- **Сетевая и многокомандная часть**

- синхронизация состояния игры между участниками;
- поддержка нескольких команд в рамках одной сессии.

- **Пользовательский интерфейс**

- интерфейс ведущего;
- интерфейс команд и игроков.

- **Хранение и обработка данных**

- данные игровых сессий;
- результаты игр и статистика.

6.3 Распределение задач по участникам

До начала реализации проекта командой был сформирован общий подход к организации работ, описанный в главе "4 Процессы разработки". Данный подход был разработан Мостовой Александрой совместно с Сухобоком Тимофеем и определил базовые принципы ведения проекта, последовательность выполнения стори и общую логику взаимодействия между участниками команды.

Обеспечение качества продукта находилось в зоне ответственности Чупрына Матвея. Совместно с Овчаровым Иваном они написали значительный объем unit- и end-to-end тестирования, что позволило своевременно выявлять и устранять ошибки как на уровне отдельных модулей, так и на уровне пользовательских сценариев.

Инфраструктурный контур проекта, включая настройку виртуальных машин, конфигурацию окружений и процесс CI/CD, разрабатывался Чуприным Матвеем и Сухобоким Тимофеем. Подробное описание архитектуры инфраструктуры, а также регламентов управления исходным кодом приведено в главе "7.1 Проектирование инфраструктуры и регламентация процессов управления исходным кодом".

Backend- и Frontend-части проекта создавались совместно большинством участников команды, поскольку проект разрабатывался с нуля и на раннем этапе не предполагал жесткого разделения на Frontend- и Backend-разработчиков. Вместе с тем можно выделить основных контрибьюторов в отдельные направления. Наибольший вклад в Backend-часть приложения внес Денисов Николай. Существенный вклад в разработку серверной части также сделали Мостова Александра и Сухобок Тимофей.

Во Frontend-часть приложения, UI/UX-дизайн и ручное тестирование наибольший вклад внесла Бакунц Арина. Дополнительно в разработке клиентской части принимали участие Мостова Александра и Денисов Николай.

7 Техническая документация

7.1 Проектирование инфраструктуры и регламентация процессов управления исходным кодом

7.1.1 Организация репозиториев

Разработка системы ведется в трех отдельных репозиториях: бэкенд-репозитории, фронтенд-репозитории и инфраструктурном репозитории.

Бэкенд-репозиторий содержит серверную часть приложения: реализацию игровой логики и правил, серверный API, механизмы хранения и обработки состояния игровых сессий, управление пользователями и ролями, а также синхронизацию данных между клиентами в реальном времени.

Фронтенд-репозиторий содержит клиентскую часть приложения: пользовательский интерфейс для участников и ведущего, визуализацию состояния игры и метрик, а также логику взаимодействия с сервером через API.

Инфраструктурный репозиторий содержит конфигурационные файлы для развертывания всей системы: конфигурацию веб-сервера Nginx, файлы Docker Compose для сборки локальных и production-окружений, скрипты деплоя и настройки CI/CD-пайплайна.

На начальном этапе проектирования рассматривалась классическая двухрепозитория модель: один репозиторий для бэкенда и один для фронтенда. Это базовое решение не требует дополнительных усилий по организации и является распространенной практикой для небольших проектов.

По мере проектирования инфраструктуры возник вопрос о размещении файлов, не относящихся ни к логике бэкенда, ни к интерфейсу фронтенда: конфигурации Nginx, файлов Docker Compose, объединяющих оба сервиса, и скриптов деплоя. Рассматривались следующие варианты:

- 1 **Разместить инфраструктурные файлы в бэкенд-репозитории:** отклонено, поскольку нарушает принцип единственной ответственности и смешивает бизнес-логику с инфраструктурными конфигурациями. Фронтенд-команда теряет прямой контроль над параметрами деплоя.
- 2 **Разместить инфраструктурные файлы во фронтенд-репозитории:** отклонено по аналогичным причинам.
- 3 **Выделить инфраструктурные файлы в отдельный репозиторий:** принято.

Инфраструктурные конфигурации выделены в отдельный репозиторий.

7.1.2 Стратегия управления ветвлением

Перед началом разработки был проведен сравнительный анализ трех стратегий управления ветвлением:

- **Git Flow** [6]: метод предполагает использование нескольких веток с долгим сроком жизни. Кроме ветки `main`, всегда содержащей стабильный код, существует `develop`, в которой ведется разработка и интеграция проекта. Для каждой задачи создается ветка `feature/*`, которая потом сливается в `develop`. Есть также ветки `release/*` и `hotfix/*`. Для нашей команды это излишне сложно. Долгоживущие ветки тормозят интеграцию и создают лишнюю работу, что противоречит бережливому производству.
- **Trunk-Based Development** [12]: разработка ведется в одной главной ветке, куда все постоянно интегрируют изменения.

Этот подход минимизирует объем незавершенной работы. Все обновления вносятся очень маленькими порциями и проходят автоматическое тестирование. Для стабилизации перед выпуском может использоваться выделение короткоживущей релизной ветки. Частые и небольшие изменения ускоряют интеграцию и снижают количество конфликтов при слиянии. Однако сейчас этот подход для нас слишком рискован. Он требует плотного покрытия автотестами: без них в ветки легко могли попасть критические ошибки, ломающие работу системы.

- **GitHub Flow** [8]: модель является более простой альтернативой Git Flow и строится на стабильной ветке `main` и временных тематических ветках (`feature-branch`), в которых ведется вся разработка. После слияния с `main` обновления сразу попадают в продакшен. Стратегия удобна для команды, однако без доработок через неё сложно отслеживать, когда большие блоки работы действительно готовы к релизу.

В результате была разработана авторская адаптация, которая основана на GitHub Flow. Процесс строится на следующих правилах:

- 1 Ветка `main` является стабильной и всегда содержит код, готовый к эксплуатации. Любое слияние в нее автоматически триггерит процесс деплоя на сервер.
- 2 Для каждой крупной задачи создается отдельная ветка `story/название-задачи`. В отличие от GitHub Flow, где ветки создаются под мелкие фичи, подход предполагает

накопление всех изменений, связанных с одной сторей, в одной ветке. Все технические задачи, декомпозированные на этапе аналитики, вливаются именно в эту ветку.

- 3 Мы сливаем ветку `story` в `main` только после того, как все задачи внутри нее выполнены, прошли код-ревью и тестирование.
- 4 Команда придерживается принципа одного потока (Single-Piece Flow) из Lean. Мы не берем новую стори в работу, пока текущая полностью не готова и не интегрирована в `main`.

Такой подход полностью соответствует логике Закона Литтла [21]: ограничивая количество одновременно открытых веток крупных задач до одной, команда минимизирует объем незавершенной работы. Это позволяет сократить время прохождения задачи по циклу и повысить предсказуемость релизов. Такой подход к управлению ветвлением помогает поддерживать высокий уровень архитектурного качества и предотвращает накопление технического долга в основной ветке разработки.

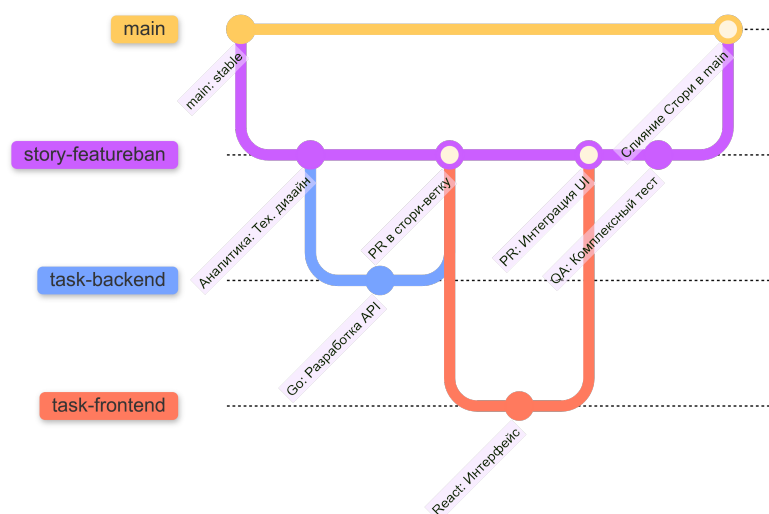


Рис. 7.1: Модифицированная стратегия ветвления на базе GitHub Flow

7.1.3 Технологический стек и автоматизация

Чтобы реализовать эту стратегию, мы выбрали стек, который стабильно работает в любых условиях:

- **Контейнеризация:** Docker позволяет собрать зависимости проекта и все необходимые файлы для развертывания. Это решает проблему несовместимости библиотек на разных машинах.
- **Сервер nginx:** Выполняет роль обратного прокси-сервера на единственном сервере проекта, обеспечивает маршрутизацию трафика.

Использование одного сервера оправдано отсутствием необходимости в горизонтальном масштабировании. Это позволяет избежать дополнительной настройки серверов.

7.1.4 Процесс релиза

Релиз цикла фиксируется навешиванием Git-тега с версией на последний коммит в ветке `main`. Это дает возможность вернуться к определенной версии в случае обнаружения критических ошибок.

Обновленный код проекта на виртуальной машине появляется с помощью команды *git pull*, выполненной на сервере, и последующим перезапуском сервиса с помощью Docker.

7.2 Страница лендинга

В начале работы мы решили заложить грамотную архитектуру макета. В рамках второй сессии команда утвердила стек технологий и реализовала страницу лендинга.

7.2.1 Сетевые маршруты и протоколы

Сервис работает на двух независимых каналах связи:

- **HTTP/1.1:** раздает статику и обрабатывает обычные запросы, где не нужно держать соединение открытым.
- **WebSocket:** держит постоянный канал для мгновенного обмена сообщениями [7], чтобы все участники видели актуальное состояние игры.

Было решено определиться с сетевыми протоколами на начальном этапе, так как в будущем предполагалась большая работа над Backend частью приложения.

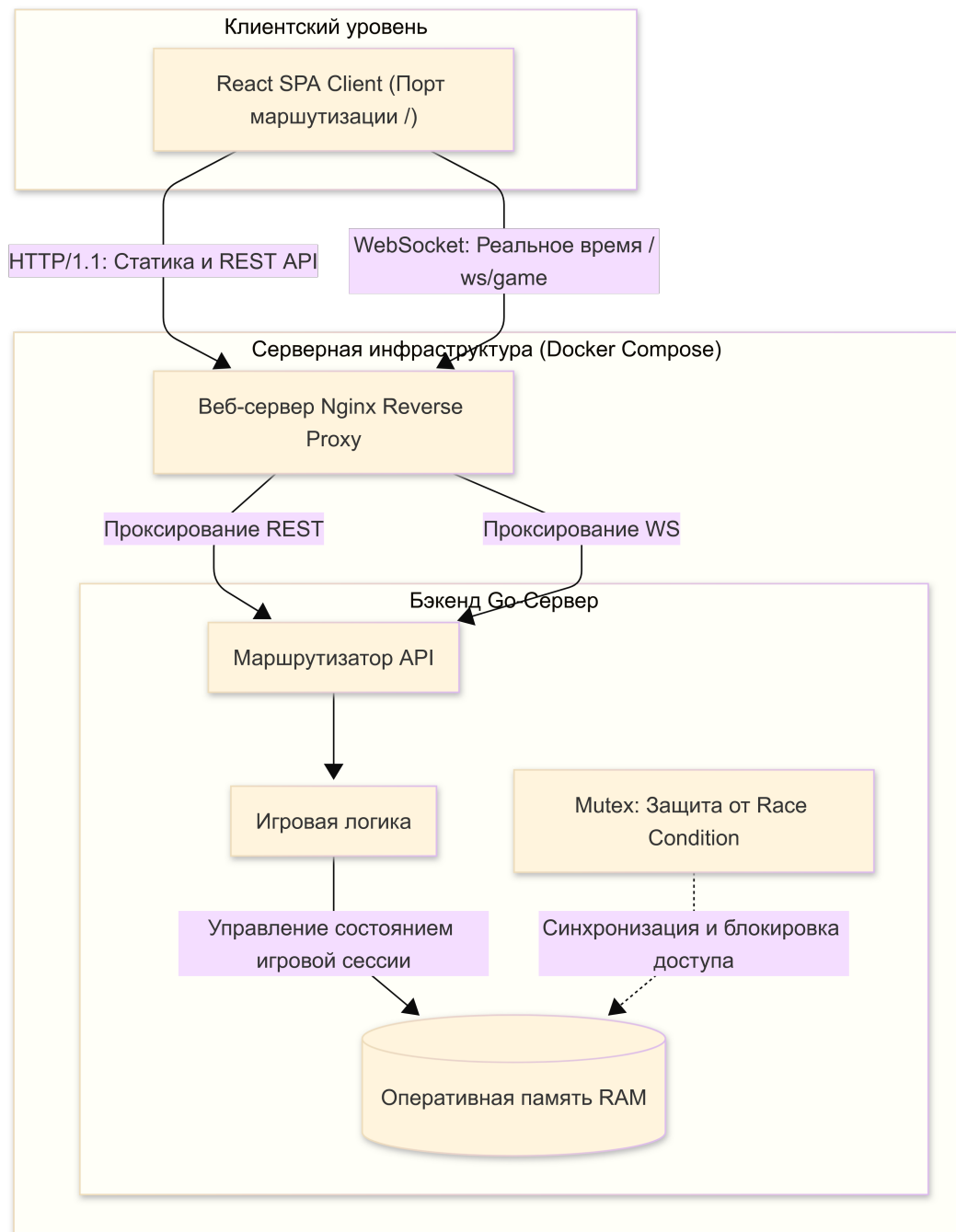


Рис. 7.2: Компонентная архитектура онлайн-симулятора и протоколы взаимодействия

7.2.2 Архитектура клиентского приложения (Frontend)

Было рассмотрено 2 варианта реализации архитектуры клиентского приложения: SPA (Single Page Application) [14] и MPA (Multi Page Application).

Было принято решение спроектировать приложение по паттерну Single Page Application (SPA) [14]. Данный подход минимизирует объем передаваемых по сети данных и обеспечивает плавность пользовательского опыта, исключая перезагрузку страниц. От MPA было решено отказаться из-за большого недостатка в виде перезагрузки страницы при каждом обновлении состояния, что разрывало бы WebSocket соединение и приводило к постоянному прерыванию процесса игры.

Интерфейс построен с помощью библиотеки React [15]. По маршруту / доступна страница лендинга приложения.

7.2.3 Архитектура серверной части (Backend)

Для серверной части проекта сначала был проведен сравнительный анализ языков программирования которые стоит использовать для написания. Команда выбирала между четырьмя языками программирования:

- **Python** — был первым рассматриваемым вариантом, так как каждый человек в команде имел много опыта программирования на нем. Несмотря на то, что с этим языком программирования работали все, и он позволяет быстро создать набросок серверной части, Python имеет ряд недостатков, среди которых: GIL (Global Interpreter Lock), который усложнит поддержку синхронизации множества игроков, более низкая производительность по сравнению с компилируемыми языками, большое потребление оперативной памяти и высокая зависимость от сторонних библиотек. Поэтому были рассмотрены другие варианты. [10]
- **Java** — Довольно популярный выбор в связи с высокой производительностью, но такая производительность была бы для нашего проекта избыточна. В то время как JVM требует много памяти и долго стартует, пользователи нашего приложения практически не заметят разницы по сравнению с другими кандидатами. Также этот язык из всех людей в команде знал один человек, что являлось для нас также недопустимым, ведь работу над Backend частью мы хотели разделить на несколько человек.
- **PHP** — Вариант был предложен в связи с его популярностью для написания веб-приложений. Однако изучив тонкости языка было обнаружено, что в PHP отсутствует

нативная поддержка WebSocket соединений, что было критическим недостатком и сразу поставило низкий приоритет выбора этого языка программирования. [18]

- **Golang** — Самый подходящий для нас вариант. Язык позволяет легко скомпилировать приложение в один бинарный файл без зависимостей. Горутины и каналы позволяют держать десятки тысяч WebSocket-соединений с минимальным расходом RAM. Также несколько участников команды знали этот язык программирования, что дало возможность распределить нагрузку по написанию серверной части проекта на несколько участников команды. [5]

Таким образом, было принято решение реализовать серверную часть на языке Go.

Для MVP версии проекта было решено хранить состояние игры в оперативной памяти сервера. Благодаря выбору Go, где каждое WebSocket-соединение потребляет минимальное количество RAM, такой подход оказался оправданным даже при большом количестве одновременных подключений. Также в Go было легко избежать проблем с рассинхронизацией при одновременном действии игроков с помощью мьютексов (Mutex), которые защищают общее состояние игры от race condition — ситуации, когда несколько горутин пытаются одновременно изменить одни и те же данные.

7.2.4 Синхронизация состояния: WebSocket и механизмы отказоустойчивости

Для синхронизации между клиентами была выбрана архитектура Event - Driven Architecture через протокол WebSocket. При подключении к эндпоинту `/ws/game` клиент инициализирует постоянное соединение. Сервер регистрирует соединение в памяти RAM с привязкой к идентификатору сессии. При любом обращении к REST API и изменении состояния игры (например, эндпоинты `/move`, `/skip_turn`), сервер инициирует broadcast-рассылку нового состояния всем активным подписчикам данной сессии [16].

Для обеспечения отказоустойчивости на клиентской стороне реализован паттерн Graceful Degradation. В случае недоступности WebSocket-соединения, клиент автоматически переходит на механизм периодического опроса (HTTP Polling) эндпоинта `/api/game/:code`. При восстановлении websocket соединения polling деактивируется.

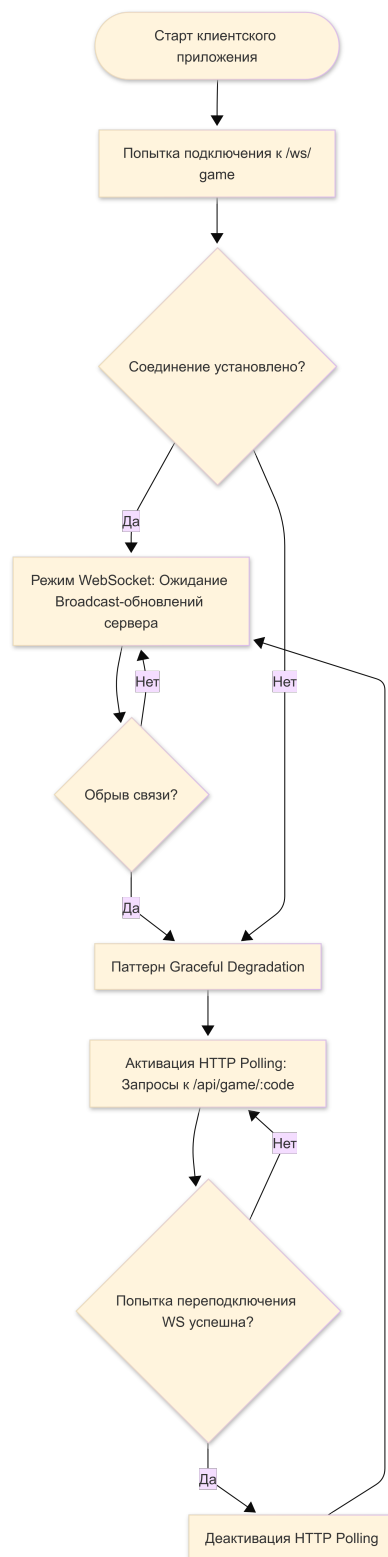


Рис. 7.3: Блок-схема алгоритма обеспечения отказоустойчивости (Graceful Degradation)

7.3 Архитектура и логика страницы игровой доски (Kanban)

Реализация интерфейса и серверной части работы игровой доски разделена на несколько уровней абстракции, обеспечивая высокую степень изоляции компонентов и мгновенную синхронизацию состояния.

7.3.1 Клиентская архитектура (Frontend)

Пользовательский интерфейс игровой страницы реализован как функциональный компонент на базе библиотеки **React** [15], с TypeScript. Для эффективного рендеринга и контроля жизненного цикла локальных данных применяются стандартные React-хуки:

- **useState** — для хранения локальных флагов (статусы загрузки, всплывающие уведомления).
- **useCallback** — для кеширования обработчиков событий (таких как загрузка состояния), защищая дочерние компоненты от лишних обновлений.
- **useRef** — для работы с Drag-and-Drop операциями и хранения промежуточного состояния перемещаемой карточки без вызова повторного рендеринга.

Чтобы избежать рассинхронизации данных был использован паттерн в котором единственный источник правды - это payload от сервера. При совершении действия (например, дропе карточки) локальное состояние не мутирует мгновенно — вместо этого отправляется WebSocket запрос к серверу, который валидирует действие по правилам игры. Только после получения подтвержденного нового состояния от сервера, React перерисовывает доску.

7.3.2 Структура игрового пространства

Игровая доска разделена на два иерархических уровня отображения:

Проектный уровень

Отображает глобальный прогресс команд и разделен на три макро-колонки. Структура данных проектов опирается на тип **ProjectCard** на сервере:

- **Сделать** — список доступных проектов. Система динамически подсчитывает количество задач (прогресс: `done_tasks / total_tasks`).
- **Интеграция** — проекты, в которых часть команд закрыла все свои задачи. Для этого этапа реализован счетчик игровых дней в интеграции (поле `days_in_integration`) и визуальная система отображения начисленных штрафов (`penalty_issued`).

- **Готово** — завершенные проекты с фиксацией дня завершения (`done_day`).

Уровень задач команд

Для каждой команды формируется собственная Kanban-доска задач, отражающая текущий рабочий процесс. Эта доска основывается на мапе состояний на сервере (задается массивом `boardStages = []string{"ready "in_progress "review "done"}`) и строго лимитирована по колонкам:

- `ready`
- `in_progress`
- `review`
- `done`

Разделение на два уровня позволяет одновременно видеть и картину продукта для фасилитатора, и микроменеджмент ресурсов внутри команд.

7.3.3 Распределенное состояние и отказоустойчивость синхронизации

Единый источник истины и блокировки

Весь актуальный стейт хранится в оперативной памяти Backend сервера. Для предотвращения состояний гонки сервер инкапсулирует каждое обращение к данным при помощи `sync.RWMutex` и `sync.Mutex`. Таким образом, любое действие, меняющее расположение карточки на доске накладывает системную блокировку структуры данных на уровне сервера.

После применения изменения, валидируемого сервером, сервер генерирует обновленное состояние и посредством broadcast-рассылки на **WebSockets** пересылает новый стейт всем активным клиентам сессии и позволяет гарантировать согласованность данных.

7.4 Игровой цикл и механика непрерывного улучшения

7.4.1 Инициализация и запуск

Игровой цикл начинается с состояния `setup`. На этом этапе ведущий инициализирует сессию с заданным максимальным количеством игровых дней (`max_days`), а игроки присоединяются к выбранным командам (REST запрос `join`). Далее фасилитатор выбирает стартовые проекты (`start_project`), формируя первоначальный бэклог продукта.

После завершения подготовительных действий сервер переводит игру в состояние `running`, и запускает первый игровой день.

7.4.2 Ежедневная продуктовая разработка

В активной фазе (**running**) каждый игровой день имитирует рабочий спринт, в течение которого разработчики тянут задачи из бэклога и передвигают их по своей Kanban-доске, применяя Drag-and-Drop интерфейс. Вся логика перемещения защищена на сервере строгими валидаторами:

- **Проверка прав:** перемещать задачи может только участник, прикрепленный к конкретной команде, предотвращая кросс-командные коллизии хода.
- **Бросок монеты:** В начале каждого дня сервер генерирует случайное событие для каждого игрока с помощью **rand**. Подобная геймификация имитирует ограничения рабочего дня программиста.
- **Контроль лимитов:** Ключевым ограничением выступают WIP-лимиты команды, встроенные структуру состояния на сервере. При попытке перетащить карточку в заполненную колонку (например, **in_progress**), сервер выбросит исключение и вернет клиенту код ошибки **409 Conflict** — "достигнут лимит WIP".

Завершение рабочего дня инициируется фасилитатором. Сервер инкрементирует счетчики времени в интеграции для зависимых проектов и сбрасывает статус ходов игроков.

7.4.3 Ретроспектива и подстройка процесса

Для внедрения в игру практики Continuous Improvement, сервер содержит триггер: каждое прохождение пяти циклов/дней $((\text{CurrentDay}-1) \% 5 == 0)$ автоматически переводит игру в состояние **Ретроспективы** (**Phase = "retro"**).

Данная фаза останавливает все возможности для перемещения тасок по доске, вынуждая команды анализировать метрики производительности и находить узкие горлышки в своем процессе разработки. Только в фазе ретроспективы команды вместе с ведущим имеют право адаптировать процесс под свои реалии.

Через эндпоинт **set_wip** фасилитатор может увеличить или уменьшить WIP-лимит на Kanban-доске конкретной команды. Это действие логируется в глобальную историю игры. По завершении обсуждения, ведущий возобновляет работу (**continue**), и игра возвращается в фазу **running**.

7.4.4 Завершение игры

Игровой цикл заканчивается по достижении установленного на старте лимита времени. Игра переходит в финальную фазу. На этом этапе доска блокируется, а команды могут проводить финальный анализ: подсчет выполненных проектов, затраченных циклов и количества штрафов. Этот процесс наглядно демонстрирует влияние лимитов незавершенной работы на общую пропускную способность команд разработки.

8 Описание применения генеративной модели

В ходе выполнения курсового проекта в качестве вспомогательного инструмента использовалась генеративная модель ChatGPT [11].

Модель использовалась, во-первых, для выделения ключевых идей в источниках литературы с последующей проверкой фактов и цитат из этих источников.

Во-вторых, для написания тестовых сценариев и нахождения граничных случаев, при которых программа могла работать некорректно. В частности, модель помогала формулировать проверки для сценариев авторизации, переходов между фазами игры, обработки WebSocket-соединений и ограничений, связанных с WIP-лимитами.

В-третьих, для краткой суммаризации уже реализованной функциональности другими участниками команды. Последнее особенно важно, так как разработка программы велась параллельно, генеративная модель позволяла быстро ознакомиться с результатами проделанной работы других участников коллектива и быстрее согласовывать отдельные фрагменты документации.

9 Заключение

В рамках курсового проекта была разработана веб-версия бизнес-игры «Мультикомандный Featureban», предназначенная для моделирования совместной работы нескольких команд в условиях ограничений по WIP, взаимных зависимостей и необходимости регулярной синхронизации состояния. В ходе выполнения работы была подтверждена возможность переноса логики настольной обучающей симуляции в онлайн-формат без потери ключевых образовательных свойств игры.

В процессе проектирования были проанализированы предметная область, сетевые протоколы, архитектурные подходы к построению клиентской и серверной частей, а также методологии организации разработки. На основе проведенного анализа была выбрана архитектура на базе SPA, реализована серверная часть на языке Go, обеспечено взаимодействие клиентов и сервера в реальном времени посредством WebSocket, а также предусмотрен резервный механизм HTTP Polling для повышения устойчивости системы при нарушении основного канала связи.

Отдельное внимание в проекте было уделено логике игровой симуляции: механике досок команд, перемещению задач, ограничению WIP, ретроспективным изменениям параметров процесса и автоматическому переходу между фазами игрового цикла. Реализованные решения позволили формализовать правила игры, сделать их воспроизводимыми в цифровой среде и обеспечить одновременную работу нескольких пользователей в рамках одной сессии.

Практическая значимость разработанного приложения заключается в возможности его использования для дистанционных корпоративных тренингов, учебных занятий и демонстрации принципов Kanban-подхода в мультикомандной среде. Полученный результат может быть использован как основа для дальнейшего развития продукта, включая расширение аналитических возможностей, улучшение пользовательского интерфейса, развитие механизмов отказоустойчивости и добавление новых сценариев взаимодействия между командами.

Список литературы

- [1] Kolb D. A. *Experiential learning: Experience as the source of learning and development*. FT Press, 2014.
- [2] Randel J. M. et al. “The effectiveness of games for educational purposes: A review of recent research”. B: *Simulation & Gaming*. (1992).
- [3] David J. Anderson. *Kanban : successful evolutionary change in your software business*. Blue Hole Press, 2010.
- [4] *Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)*. URL: <https://creativecommons.org/licenses/by-sa/4.0/> (дата обр. 03.02.2026).
- [5] Alan AA Donovan и Brian W Kernighan. *The Go programming language*. Addison-Wesley Professional, 2015.
- [6] Vincent Driessen. *A successful Git branching model*. 2010. URL: <https://nvie.com/posts/a-successful-git-branching-model/> (дата обр. 10.03.2026).
- [7] Ian Fette и Alexey Melnikov. *The websocket protocol*. Тех. отч. RFC 6455, December, 2011.
- [8] GitHub. *Understanding the GitHub flow*. 2023. URL: <https://docs.github.com/en/get-started/using-github/github-flow> (дата обр. 10.03.2026).
- [9] Cox J. Goldratt E. M. *The goal: a process of ongoing improvement*. Routledge, 2016.
- [10] Ozsvald I. Gorelick M. *High performance python*. O'Reilly Media, Inc., 2025.
- [11] *GPT*. URL: <https://chatgpt.com/> (дата обр. 29.05.2026).
- [12] Paul Hammant. *Trunk-Based Development*. 2013. URL: <https://trunkbaseddevelopment.com/> (дата обр. 10.03.2026).
- [13] Kerridge C. Loon M. Evans J. “Learning with a strategic management simulation game: A case study”. B: *The International Journal of Management Education*. (2015).
- [14] Ali Mesbah и Arie van Deursen. “Architectural implications of single page web applications”. B: *Proceedings of the 13th international conference on software engineering*. 2007.
- [15] Inc. Meta Platforms. *React: A JavaScript library for building user interfaces*. 2024. URL: <https://react.dev/> (дата обр. 27.03.2026).
- [16] Victoria Pimentel и Bradford G Nickerson. “Communicating and displaying real-time data with websocket”. B: *IEEE Internet Computing* 16.4 (2012), с. 45—53.

- [17] Sutherland J. Schwaber K. *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Scrum Alliance, 2020.
- [18] MacIntyre P. Tatroe K. *Programming PHP: Creating dynamic web pages*. O'Reilly Media, Inc., 2020.
- [19] Jones D. T. Womack J. P. *Lean thinking: banish waste and create wealth in your corporation*. New York: Free Press, 2003.
- [20] Майк Барроуз. *Канбан Метод: улучшение системы управления*. Альпина паблишер, 2023.
- [21] Малоизвестные факты о Законе Литтла: как обеспечить предсказуемость рабочего процесса. URL: <https://neogenda.com/blog/little-law-prediction> (дата обр. 13.01.2026).
- [22] Мультикомандный Featureban. URL: <https://neogenda.com/multifeatureban> (дата обр. 13.01.2026).